MASTER PROMPT: BUILD A PRODUCTION-READY MLH-INSPIRED OPPORTUNITY & INTERVIEW INTELLIGENCE PLATFORM

Act as a Principal Full-Stack Engineer, Lead UI/UX Architect, and senior React/Node.js developer.

Your task is to build a complete, functional, responsive full-stack web application for discovering hackathons, internships, fresher jobs, and preparing for their selection processes.

The product must combine the visual quality, playful illustrated design language, strong typography, generous spacing, and polished interactions of the Major League Hacking website with my specified dark navy and cyan design system.

Reference website: https://www.mlh.com/

Project name: MLH-Inspired Opportunity & Prep Intelligence Platform.

Primary objective: Deliver a working application with complete source code, a functional frontend, an Express REST API, MongoDB integration, realistic seed data, and a high-quality interview preparation experience.

Use the attached PDF as an additional requirements document. Read the entire PDF before implementing anything. If the PDF and this prompt conflict, preserve the explicit technical requirements in this prompt and clearly document any remaining conflict.

---

1. CRITICAL VISUAL REQUIREMENT: RECREATE THE MLH-INSPIRED DESIGN LANGUAGE

Before coding, inspect the reference website, particularly its initial viewport, hero section, right-side illustration, header, typography, spacing, and responsive behavior.

Do not create a generic admin dashboard, generic SaaS landing page, or basic grid of dark cards.

The site must feel like a polished developer-community platform inspired by MLH's visual storytelling.

Hero illustration — highest priority

The hero section must have a substantial, visually interesting illustration on the RIGHT side of the main headline and description.

Study the isometric hero illustration visible on the MLH homepage and recreate its overall visual character using an original, locally implemented illustration.

The illustration should convey software development, hackathons, and technical opportunities through a cohesive isometric composition. It may incorporate:

- Floating browser windows and code editor panels.
- Small developer workstations, laptops, or computer screens.
- Colorful geometric platforms and layered shapes.
- Floating code symbols, terminal snippets, and UI elements.
- Small decorative elements that suggest a vibrant developer ecosystem.
- Subtle depth, shadows, highlights, and controlled use of accent colors.

Use an original SVG illustration, carefully constructed CSS artwork, or a combination of both. Prefer inline SVG or a dedicated local SVG asset so the illustration remains crisp, lightweight, and reliable.

Do not substitute the illustration with a generic emoji, a few floating icons, a stock photograph, or an unstyled collection of rectangles.

Do not claim to have reproduced the original illustration exactly if the original asset is unavailable. Instead, create a distinctive original illustration that captures its composition, dimensionality, playful developer

aesthetic, and visual balance.

The illustration must:
- Occupy approximately 45–55% of the desktop hero's visual width.
- Balance the headline and CTA area on the left.
- Have carefully controlled decorative details rather than visual clutter.
- Support subtle CSS animations, such as gentle floating or parallax-like movement.
- Respect `prefers-reduced-motion`.
- Scale cleanly on tablets and mobile devices.
- Never overflow the viewport or obscure the headline.
- Render without external image dependencies.

Use the original MLH brand assets only if their permitted usage is clear. This project is MLH-inspired, not an official MLH product. Do not falsely imply affiliation or endorsement.

Visual design system

Implement these exact base colors:

- Page background: `#0b1426`
- Card and surface background: `#111b2e`
- Hover border: `#1f2d48`
- Announcement banner: `#dc2626`
- Primary cyan: `#00f0ff`
- Secondary cyan: `#00b4d8`
- React tag: `#38bdf8`
- Node.js tag: `#22c55e`
- MongoDB tag: `#10b981`
- Primary text: `#ffffff`
- Secondary text: `#94a3b8`

Define reusable CSS variables for these colors and use Tailwind utilities where appropriate.

Use a modern sans-serif typeface with strong, readable headings, comfortable line height, and carefully balanced font weights.

Use cyan for primary actions, interactive highlights, and selected states. Use red sparingly for the announcement banner and important urgency indicators.

Do not turn every element cyan. Preserve a disciplined visual hierarchy.

Visual polish requirements

- Use consistent content widths and spacing.
- Use rounded corners deliberately, not excessively.
- Give cards subtle borders and restrained shadows.
- Include tasteful hover transitions and focus states.
- Use clear heading sizes and readable body text.
- Include subtle background gradients or geometric details where appropriate.
- Maintain strong text contrast.
- Use Lucide icons through `lucide-react` where appropriate.
- Avoid emoji as substitutes for professional icons.
- Avoid excessive glassmorphism, neon glow, and unnecessary gradients.
- Avoid horizontal scrolling.
- Ensure the design looks finished at 1440px, 1024px, 768px, and 375px viewport widths.

---

2. REQUIRED TECHNOLOGY STACK

Frontend:

- React
- Vite
- React Router if routing is needed
- Tailwind CSS
- Custom CSS for the hero illustration and advanced styling
- Axios
- Lucide React

Backend:
- Node.js
- Express
- MongoDB
- Mongoose
- dotenv
- cors
- Helmet
- Express rate limiting
- Appropriate request validation

Use compatible, stable package versions and include all required dependencies in the appropriate `package.json` files.

Do not introduce an unnecessary framework or replace the requested stack.

---

3. EXACT PROJECT DIRECTORY STRUCTURE

Preserve the following required files and locations:

```
my-mlh-platform/
├── client/
│   ├── index.html
│   ├── vite.config.js
│   ├── tailwind.config.js
│   └── src/
│       ├── main.jsx
│       ├── App.jsx
│       ├── index.css
│       ├── components/
│       │   ├── TopBanner.jsx
│       │   ├── Navbar.jsx
│       │   ├── HeroSection.jsx
│       │   ├── FilterAndSearch.jsx
│       │   ├── OpportunityGrid.jsx
│       │   ├── OpportunityCard.jsx
│       │   ├── PrepDrawerModal.jsx
│       │   ├── TechStackBadge.jsx
│       │   └── Footer.jsx
│       ├── services/
│       │   └── api.js
│       └── data/
│           └── mockData.js
└── server/
    ├── package.json
    ├── server.js
    ├── .env
    ├── config/
    │   └── db.js
    ├── models/
    │   └── Opportunity.js
    ├── controllers/
    │   └── opportunityController.js
    └── routes/
        └── opportunityRoutes.js
```

You may add supporting files when they materially improve the implementation, such as a README, a database seeder, a dedicated SVG illustration, or tests. Never omit any required file.

Do not silently move files into a different architecture.

---

4. ANNOUNCEMENT BANNER

File: `client/src/components/TopBanner.jsx`

Build a responsive announcement bar at the top of the page.

Use the requested red background `#dc2626`.

Text:

"LIVE OPPORTUNITIES: Hackathons, Internships & Fresher Jobs | Updated Daily"

Include a right-aligned link:

"View MLH Guidelines →"

Link to a valid official MLH guidelines or resources page. Do not invent a URL.

Ensure the text wraps or adapts gracefully on mobile. The banner must not cover the navbar or hero.

---

5. NAVIGATION BAR

File: `client/src/components/Navbar.jsx`

Build a professional dark navy navigation bar.

Left:
- A compact, original text-based brand mark reading `MLH`.
- Supporting text: `OPPORTUNITIES & PREP`.
- Do not use the official MLH logo in a misleading manner.

Center:
- Hackathons
- Internships
- Fresher Jobs
- Prep Intelligence

Right:
- A `Post Opportunity` CTA with a cyan border and restrained glow.

Requirements:
- Responsive mobile navigation.
- Accessible menu button.
- Working anchor navigation or React Router links.
- Sticky positioning only if it improves usability.
- Keyboard-accessible controls.
- Clear active and hover states.

The `Post Opportunity` button must open a functional submission form or modal. Validate the form and connect it to the backend POST endpoint. Never present an inert button as a finished feature.

---

6. HERO SECTION

File: `client/src/components/HeroSection.jsx`

Headline:

"Discover Opportunities. Master The Intelligence."

Supporting text:

"Find top hackathons, internships, and entry-level software engineering roles. Get step-by-step blueprints, study notes, and technical interview questions to crack them."

Include:
- A primary CTA to explore opportunities.
- A secondary CTA to explore preparation resources.
- Three technology badges: React.js, Node.js, MongoDB.
- The substantial original SVG/CSS illustration on the right.
- A small trust or value statement, if it improves the composition.

The hero should feel like a carefully art-directed homepage rather than a text block followed by a row of cards.

Use a two-column desktop layout and a deliberate stacked mobile layout.

Do not place the illustration below the fold on standard desktop screens.

---

7. TECHNOLOGY BADGES

File: `client/src/components/TechStackBadge.jsx`

Create a reusable badge component with support for:

- React.js
- Node.js
- MongoDB
- Python
- JavaScript
- TypeScript
- Express
- Other technology tags returned by the API.

Apply the specified technology colors where appropriate.

Unknown tags should use a neutral style rather than crashing or producing undefined styles.

Badges must remain legible on the dark background.

---

8. OPPORTUNITY SEARCH AND FILTERS

File: `client/src/components/FilterAndSearch.jsx`

Implement a functional search and filtering interface.

Categories:
- All Opportunities
- Hackathons
- Internships
- Fresher Jobs

Features:
- Search by opportunity title.
- Search by organization or company name.
- Search by technology tags.
- Category filter pills.
- Active category styling.

- Clear search action.
- Loading, empty, and error states.
- Responsive layout.
- Debounced search if appropriate.

Search must work with the backend API, not merely hide cards in a static array.

Changing a category must update the results correctly. Search and category filtering must work together.

The page should not reload on every search keystroke.

---

9. OPPORTUNITY GRID AND CARDS

Files:
- `client/src/components/OpportunityGrid.jsx`
- `client/src/components/OpportunityCard.jsx`

Use a responsive grid equivalent to:

`grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6`

Each card should include:

- Opportunity category badge.
- Opportunity title.
- Company or organization.
- Location and work mode.
- Technology badges.
- Application deadline or event date.
- A concise description.
- A `View Opportunity` action.
- A `Crack Prep Intelligence` action.

Use visually differentiated category badges for hackathons, internships, and fresher jobs.

Design the cards with the specified surface color, thin borders, deliberate spacing, and subtle hover transitions.

When the user selects `Crack Prep Intelligence`, open `PrepDrawerModal.jsx` with the correct opportunity data.

When the user selects `View Opportunity`, display the complete details and provide an external application or registration link if one exists.

Clearly label expired opportunities. Do not fabricate deadlines, company partnerships, application links, or live availability.

Provide at least 12 realistic demonstration records across all three categories. Label demonstration content as sample data and avoid claiming that it represents verified current openings.

---

10. INTERVIEW PREPARATION DRAWER

File: `client/src/components/PrepDrawerModal.jsx`

This is a core product feature, not a decorative modal.

Create a polished slide-over drawer on desktop and an accessible full-screen or near-full-screen panel on mobile.

Display the selected opportunity's title, organization, category, and relevant preparation material.

Implement three navigable sections or tabs.

A. Preparation Roadmap

Provide a practical four-week roadmap with weekly goals, daily study tasks where useful, hands-on practice, and measurable milestones.

Example:
- Week 1: Fundamentals and prerequisite concepts.
- Week 2: Core technical concepts and implementation.
- Week 3: Projects, problem-solving, and mock interviews.
- Week 4: Revision, timed practice, and final preparation.

Customize the roadmap according to the opportunity category and role.

For hackathons, focus on ideation, team formation, rapid prototyping, architecture, pitching, and judging criteria.

For internships and fresher jobs, focus on programming, data structures and algorithms, CS fundamentals, role-specific technologies, projects, and interview practice.

B. Core Technical Focus

For JavaScript, React, Node.js, Express, and MongoDB, include relevant topics such as:

- JavaScript execution model, closures, promises, and async/await.
- React components, hooks, state management, and rendering.
- Node.js event loop and asynchronous I/O.
- Express middleware, routing, error handling, and REST API design.
- MongoDB schema design, indexing, aggregation pipelines, and query optimization.
- Authentication, authorization, security, and API validation.
- Testing, debugging, and performance optimization.

Display topics in readable sections with concise explanations or expandable details.

Do not dump a long unstructured list of buzzwords.

C. Screening & Interview Q&A

Provide role-relevant questions with clear, technically accurate answers.

Include:
- Programming and coding questions.
- Data structures and algorithms.
- JavaScript and framework questions.
- Backend and database questions.
- API and system-design fundamentals appropriate to the role.
- Behavioral questions.
- Hackathon-specific evaluation and pitching questions where relevant.

Use sample questions tailored to the selected opportunity rather than identical questions for every card.

Add useful interactions, such as expanding answers, checking preparation tasks, and switching between tabs.

Support closing the drawer with a close button, Escape key, and backdrop click. Prevent background interaction while open, manage focus appropriately, and restore focus when closed.

Ensure every interactive element works.

---

11. FRONTEND API SERVICE

File: `client/src/services/api.js`

Use Axios with a configurable API base URL.

Default development URL:

`http://localhost:5000/api`

Use a Vite environment variable such as `VITE_API_BASE_URL` to configure the base URL.

Implement:

- `fetchOpportunities(category, searchQuery)`
- `fetchOpportunityById(id)`
- `createOpportunity(payload)`

Handle HTTP errors, timeouts, malformed responses, and network failures appropriately.

Keep API requests outside presentational components wherever practical.

---

12. MOCK DATA AND OFFLINE DEVELOPMENT

File: `client/src/data/mockData.js`

Create at least 12 complete sample opportunities.

Each record should contain appropriate values for:

- ID.
- Title.
- Company or organization.
- Category.
- Location.
- Work mode.
- Tags.
- Description.
- Deadline or event date.
- External application URL where applicable.
- Preparation roadmap.
- Technical topics.
- Sample questions and answers.

Include multiple examples of each opportunity category.

Use realistic, clearly identified sample records. Do not pretend they are live, verified opportunities.

The application must remain useful if MongoDB is unavailable during development.

Implement a controlled fallback to local mock data when the API is unreachable. Show a clear visual notice that demonstration data is being used.

Do not silently swallow API errors or show a misleading success message.

When the API is working, use backend data as the primary source.

---

13. MONGOOSE DATABASE MODEL

File: `server/models/Opportunity.js`

Create a robust Mongoose schema with:

- `title`: required string.
- `company`: required string.
- `category`: required enum of `Hackathon`, `Internship`, `Fresher Job`.
- `location`: string.
- `workMode`: suitable enum or validated string.

- `tags`: array of strings.
- `description`: string.
- `deadline`: date or nullable date.
- `eventDate`: nullable date.
- `applicationUrl`: validated URL or empty value.
- `prepIntelligence`: nested object containing `roadmap`, `technicalTopics`, and `sampleQuestions`.
- `isDemo`: boolean to identify demonstration records.
- Created and updated timestamps.

Use structured subdocuments for questions and answers where appropriate.

Add useful indexes for category, title, company, tags, and search.

Avoid unnecessary duplication of database fields.

---

14. DATABASE CONNECTION

File: `server/config/db.js`

Implement a Mongoose connection function that:

- Reads the MongoDB connection string from environment variables.
- Logs a successful connection.
- Reports connection failures clearly.
- Handles disconnection and reconnection events sensibly.
- Does not expose credentials in logs.

Support MongoDB Atlas and a local MongoDB instance.

If the database is unavailable, the API must report the appropriate state and the frontend must activate its controlled demonstration-data fallback.

Never silently claim that records were saved when no database write occurred.

---

15. EXPRESS CONTROLLER

File: `server/controllers/opportunityController.js`

Implement:

`getOpportunities`

- Accept category and search query parameters.
- Support pagination using page and limit.
- Filter by category.
- Search title, company, and tags.
- Use safe, escaped regular expressions or an appropriate text-search strategy.
- Return consistent JSON responses with pagination metadata.
- Validate query parameters.
- Handle database errors.

`getOpportunityById`

- Validate the identifier.
- Return the complete opportunity, including preparation intelligence.
- Return 404 when the record does not exist.
- Handle malformed IDs and database errors correctly.

`createOpportunity`

- Validate and sanitize submitted data.
- Require the essential opportunity fields.
- Validate category, tags, dates, URLs, and preparation material.
- Return appropriate HTTP status codes.
- Never report success before the database write succeeds.

Use a consistent response structure and appropriate error messages.

---

16. EXPRESS ROUTES

File: `server/routes/opportunityRoutes.js`

Implement:

- `GET /api/opportunities`
- `GET /api/opportunities/:id`
- `POST /api/opportunities`

Use the controller functions and apply appropriate validation middleware.

Keep the routes thin and the business logic in the controller.

---

17. SERVER CONFIGURATION

Files:
- `server/server.js`
- `server/package.json`
- `server/.env`

Configure Express with:

- JSON request parsing and sensible request-size limits.
- CORS restricted to configured frontend origins.
- Helmet security headers.
- Rate limiting.
- Centralized error handling.
- A health endpoint such as `GET /api/health`.
- API routes mounted under `/api/opportunities`.
- A configurable port defaulting to `5000`.
- Graceful shutdown handling.

Use environment variables for database credentials, port, and allowed origins.

Provide a complete `server/.env.example` as an additional file.

The `.env` file must contain only safe placeholders. Never embed actual credentials or commit secrets.

Do not hard-code production credentials or permit unrestricted production CORS.

Ensure the server starts predictably when the database is available and reports its readiness accurately.

---

18. VITE, TAILWIND, AND FRONTEND ENTRY POINT

Implement the following files completely:

`client/index.html`
Provide correct metadata, a page title, viewport settings, and the React mount element.

`client/vite.config.js`

Configure Vite and React correctly, including development API proxying if useful.

`client/tailwind.config.js`
Configure Tailwind content paths and the specified theme colors.

`client/src/main.jsx`
Mount React correctly and import global styles.

`client/src/App.jsx`
Compose the banner, navbar, hero, filters, opportunity grid, preparation drawer, and footer.

Implement the actual data-fetching and UI state management required to connect these components.

`client/src/index.css`
Implement Tailwind directives, CSS variables, global typography, background styles, reusable interaction states, animation keyframes, responsive refinements, and reduced-motion support.

---

19. FOOTER

File: `client/src/components/Footer.jsx`

Build a polished footer consistent with the rest of the application.

Include navigation to opportunity categories, preparation resources, and appropriate external resources.

Provide a clear note that this is an independent, MLH-inspired project and is not an official MLH product.

Do not fabricate partnerships or affiliations.

---

20. PRODUCTION QUALITY AND ACCESSIBILITY

The application must include:

- Responsive layouts.
- Semantic HTML.
- Accessible form labels.
- Keyboard navigation.
- Visible focus states.
- Proper modal and drawer accessibility.
- Loading indicators.
- Empty states.
- Error states.
- Form validation.
- Secure API input handling.
- Reusable components.
- Maintainable naming and folder organization.
- No unnecessary prop drilling where a simple alternative exists.
- No broken imports.
- No missing dependencies.
- No console errors during normal usage.
- No nonfunctional placeholder buttons.
- No misleading claims about live opportunity data.

Do not build unnecessary authentication, payment systems, or AI API integrations unless explicitly required by the attached PDF.

If an actual AI model is not configured, provide useful curated preparation content instead of pretending that AI-generated responses are live.

21. REQUIRED DELIVERY FORMAT

Write the COMPLETE, UNABRIDGED CODE for every required file in the specified directory structure.

Do not provide pseudocode.

Do not omit CSS because it is long.

Do not use placeholder comments such as:
- `// TODO: implement`
- `/* Add styles here */`
- `// Remaining code omitted`
- `...`

Do not replace complete implementations with explanations of what the developer should do.

For each file, provide:

1. Its exact relative path.
2. Its complete contents in an appropriate fenced code block.
3. All necessary imports, exports, configuration, and error handling.

Start with a concise architecture overview and the final directory tree.

Then provide every file, grouped into frontend, backend, and setup documentation.

If the response length is insufficient, split the delivery into clearly numbered parts. Stop only at a file boundary, state which files have been completed and which remain, and continue with the remaining complete files when prompted. Never truncate a file or falsely claim that all files have been delivered.

Include a complete README containing:
- Prerequisites.
- Installation commands.
- Environment configuration.
- MongoDB setup.
- How to seed demonstration records.
- How to start the backend.
- How to start the frontend.
- How to verify the API.
- How to troubleshoot common errors.
- How the mock-data fallback behaves.

Provide exact commands for Windows PowerShell as well as a brief macOS/Linux equivalent.

---

22. TESTING AND ACCEPTANCE CRITERIA

Before declaring the project complete, check the implementation against these criteria:

1. The frontend builds successfully.
2. The backend starts without syntax or import errors.
3. MongoDB connection handling works as documented.
4. The health endpoint returns a meaningful status.
5. Opportunity listing works.
6. Category filtering works.
7. Search by title, company, and technology works.
8. Pagination works.
9. Opportunity detail retrieval works.
10. Opportunity submission validates input and persists records when the database is available.
11. The preparation drawer opens with the selected opportunity's data.

12. The roadmap, technical topics, and Q&A are navigable.
13. The navigation links and CTAs perform their intended actions.
14. The app displays useful sample content if the backend is unavailable.
15. The desktop hero includes a substantial original illustration on the right.
16. The application remains usable on mobile.
17. Keyboard navigation and reduced-motion preferences work.
18. No required source file is missing.
19. No credentials or secrets are exposed.
20. The documentation accurately reflects the implementation.

Do not claim that tests were run unless they were actually executed. If execution is unavailable, state which checks remain unverified and provide the commands needed to run them.

FINAL INSTRUCTION

Prioritize visual quality, completeness, correctness, and real functionality over producing a short response.

First inspect the PDF and reference website. Then establish the design system and architecture, implement the application, validate the source code, and deliver all files.

The finished result should look like a carefully designed developer-community product inspired by MLH, with a compelling illustrated hero, a coherent dark navy/cyan theme, useful opportunity discovery, and substantial interview preparation intelligence—not a generic template with recolored cards.